
Documentation: base_monitor.py

Overall Purpose: The Blueprint for All Monitors

Think of this BaseMonitor class as a **blueprint** or a **template** for any monitor you want to create.

Imagine you're building a team of specialist workers (a file watcher, a network watcher, etc.). Instead of training each worker from scratch, you give them all a standard employee handbook. The handbook tells them:

- How to report findings (log_alert).
- What their daily work schedule is (run_wrapper).
- That they must know their own job title (get_name).
- That they must know what their specific job task is (run).

The BaseMonitor is this "employee handbook." Any new monitor we create (like file_monitor.py) will **inherit** from BaseMonitor, meaning it automatically gets all the standard functionality and is forced to follow the essential rules.

This approach has two huge benefits:

1. **Avoids Repeating Code:** We write the common logic for starting, stopping, and logging in one place. Every new monitor gets this for free without us copying and pasting code.
2. **Enforces Consistency:** It guarantees that every monitor will have the same basic structure, making the whole EDR agent more predictable and easier to manage.

Code Breakdown

1. The BaseMonitor Class

This is the main blueprint that defines the shared attributes and behaviors for all monitors.

`__init__(self, agent_config: dict, log_queue: Queue, shutdown_event: Event)` (The Constructor)

- This `__init__` method is the setup function. When the main EDRAgent creates a monitor, it "hands it" three essential tools:
 - `agent_config`: The complete configuration dictionary loaded from `config.json`.
 - `log_queue`: The shared message queue. This is the monitor's **only way** to communicate back to the main agent process.
 - `shutdown_event`: The shared signal that the main agent will use to tell the monitor when it's time to stop running.
- It then sets up a few helpful attributes for the monitor to use:
 - `self.monitor_config`: A convenient shortcut to the monitor's *specific* section in the config file. For example, for the file integrity monitor, this would be the dictionary under the "file_integrity" key.
 - `self.interval`: The number of seconds the monitor should wait between each check. It gets this from the agent section of the config.

`get_name(self) -> str`

- This is a rule defined in the blueprint. It says, "Every monitor that is built from me **must** have a `get_name` method that returns its own unique name (like 'file_integrity')." **raise NotImplementedError**: This is how the rule is enforced. If you create a new monitor class but forget to implement this method, the program will immediately crash with this error, telling you exactly what you need to fix.

`log_alert(...)`

- This is a helper function that makes it easy for any monitor to send a properly formatted alert back to the main agent.
- Instead of manually creating a dictionary every time, a monitor can simply call `self.log_alert(...)` with the details.
- This ensures every alert sent through the `log_queue` has a consistent structure, which is very important for the main agent that has to process them.
- Notice the action parameter. This is how a monitor can request the EDRAgent to perform an active response, like "kill_process" or "block_ip".

`run_wrapper(self)`

- This is the **engine** that drives every monitor. The main EDRAgent starts this function in a separate process for each monitor.
- It controls the entire **lifecycle** of the monitor:
 1. **Log Start:** It first sends an informational message to the queue announcing that it has started.
 2. **One-Time Setup:** It calls `self.start()`. This is for any setup code a monitor needs to run only once at the very beginning.
 3. **Main Loop:** It enters a while loop that continues as long as the `shutdown_event` has not been triggered by the main agent.
 4. **Check and Run:** Inside the loop, it first checks if its "enabled" flag is true in the config. If it is, it calls the `self.run()` method, which contains the monitor's core logic.
 5. **Sleep:** It then pauses for the configured interval before the next loop. This prevents the monitor from using 100% of the CPU.
 6. **One-Time Cleanup (finally):** When the loop ends (because the shutdown signal was received), the finally block is guaranteed to execute. It calls `self.stop()` for any cleanup tasks and sends a final message saying it's stopping.

`run(self)`

- This is another rule, just like `get_name`. The blueprint says, "I will provide the engine that calls `run()` on a schedule, but you **must** provide the `run()` method that contains your specific monitoring logic."
- The `NotImplementedError` ensures that a new monitor won't work until the developer has written the code for what it's supposed to be checking.

start(self) and stop(self)

- These are optional methods for setup and cleanup. They are empty in the base class, but a specific monitor can implement them if needed.
- **Example start() use:** A file integrity monitor could use start() to scan the target directories and create a baseline of file hashes when it first boots up.
- **Example stop() use:** A monitor that creates temporary files could use stop() to make sure those files are deleted cleanly upon shutdown..